

國立中央大學

資訊工程研究所

碩士論文

SLM 驅動的代理：以 Grafana 為例的可視化導向自然語言互動介面

SLM-Powered Agent for Visualization-Oriented Natural Language Interface: A Case Study on Grafana

研究生：童伯堯

指導教授：林家瑜 博士

中華民國一百一十四年四月

國立中央大學圖書館學位論文授權書

填單日期：____/____/____

2019.9 版

授權人姓名		學 號	
系所名稱		學位類別	☐ 碩士 ☐ 博士
論文名稱		指導教授	

學位論文網路公開授權

授權本人撰寫之學位論文全文電子檔：

• 在「國立中央大學圖書館博碩士論文系統」.

() 同意立即網路公開

() 同意 於西元____年____月____日網路公開

() 不同意網路公開，原因是：_____

• 在國家圖書館「臺灣博碩士論文知識加值系統」

() 同意立即網路公開

() 同意 於西元____年____月____日網路公開

() 不同意網路公開，原因是：_____

依著作權法規定，非專屬、無償授權國立中央大學、台灣聯合大學系統與國家圖書館，不限地域、時間與次數，以文件、錄影帶、錄音帶、光碟、微縮、數位化或其他方式將上列授權標的基於非營利目的進行重製。

學位論文紙本延後公開申請 (紙本學位論文立即公開者此欄免填)

本人撰寫之學位論文紙本因以下原因將延後公開

• 延後原因

() 已申請專利並檢附證明，專利申請案號：

() 準備以上列論文投稿期刊

() 涉國家機密

() 依法不得提供，請說明：_____

• 公開日期：西元____年____月____日

※繳交教務處註冊組之紙本論文(送繳國家圖書館)若不立即公開，請加填「國家圖書館學位論文延後公開申請書」

研究生簽名：_____ 指導教授簽名：_____

*本授權書請完整填寫並親筆簽名後，裝訂於論文封面之次頁。

國家圖書館學位論文延後公開申請書

Application for Embargo of Thesis/Dissertation

申請日期：民國____年____月____日

Application Date: ____/____/____(YYYY/MM/DD)

申請人姓名 Applicant Name		學位類別 Graduate Degree	☐ 碩士 Master ☐ 博士 Doctor	畢業年月 Graduation Date (YYYY/MM)	民國____年____月____/
學校名稱 University			系所名稱 School/Department		
論文名稱 Thesis / Dissertation Title					
延後公開原因 Reason for embargo	☐ 涉及機密 Contains information pertaining to the secret. ☐ 專利事項，申請案號： Filing for patent registration. Registration number: ☐ 依法不得提供，請說明： Withheld according to the law. Please specify.				
申請項目 Options	☐ 紙本論文延後公開 Delay public access to the printed copies of my thesis, but leave the online bibliographic record open to the public.			☐ 書目資料延後公開 Delay public access to online bibliographic record of my thesis.	
公開日期 Delayed Until	民國____年____月____日 ____/____/____(YYYY/MM/DD)			☐ 不公開 Prohibited from public access.	

申請人簽名：

Applicant Signature: _____

指導教授簽名：

Advisor Signature: _____

學校認定/審議單位章戳：

Seal of the Authorization Institute: _____

【說明】

- 依教育部107年12月5日臺教高(二)字第1070210758號函及109年3月13日臺教高通字第1090027810號函，請據實填寫本申請書並檢附由學校認定或審議單位認定之證明文件，經由學校向本館提出申請，無認定或審議單位章戳者退回學校處理。
- 論文尚未送交國家圖書館，請於提送論文時，夾附親筆簽名申請書1份。
- 論文已送達國家圖書館，請將親筆簽名申請書一式2份掛號郵寄10001臺北市中山南路20號國家圖書館館藏發展及書目管理組，並於信封註明「學位論文延後公開申請書」。
- 本館保存之學位論文依學位授予法應提供公眾於館內閱覽紙本，或透過獨立設備讀取電子資料檔，二者依表單填寫日期公開。

【Notes】

- Please fill in all blanks and attach the certification documents approved by the university and apply through the university. The application form will not be accepted for processing until all information, signatures, and stamps are included.
- If the thesis or dissertation is not yet submitted to the NCL, please attach the signed application form to the thesis or dissertation.
- If the thesis or dissertation has been submitted to the NCL, please send a registered letter with 2 copies of the signed application form attached. The letter should be addressed to "Collection Development Division", National Central Library with a note in the envelope indicating "Application for delay of public release" to the following address. No.20, Zhongshan S. Rd., Zhongzheng District, Taipei City 10001, Taiwan (R.O.C.)
- The delayed date of printed copies and the independent viewing equipment will synchronize.

(申請者免填，以下由國家圖書館填寫 For Internal Use)

承辦單位_館藏組：_____日期/處理狀況：

典藏地：_____登錄號：_____索書號：

會辦單位_知服組：_____日期：_____ ☐移送並註記，原上架日期：

論文系統：_____日期：

國立中央大學碩士班研究生
論文指導教授推薦書

_____學系/研究所_____研究生所提之論文_____

係由本人指導撰述，同意提付審查。

指導教授_____ (簽章)

____年____月____日

國立中央大學碩士班研究生
論文口試委員審定書

_____學系/研究所_____研究生
所提之論文

經本委員會審議，認定符合碩士資格標準。

學位考試委員會召集人 _____

委 員 _____

中 華 民 國 年 月 日

摘要

傳統的視覺化導向自然語言互動介面（V-NLI）主要依賴下拉式選單來產生圖表，這種方式相較於自然語言輸入不夠直觀。因此，近年來越來越多研究開始引入大型語言模型（LLM），透過自然語言輸入來生成視覺化圖形。儘管 LLM 在視覺化領域表現出色，但隨著模型複雜度提升，LLM 對 GPU 與硬體資源的需求也隨之增加，使得本地部署變得困難，這促使輕量化的小型語言模型（SLM）成為替代方案。

雖然 SLM 具備輕量化優勢，但在生成視覺化圖形時仍面臨諸多挑戰，例如產生的圖形參數不符合使用者需求，或在相同輸入下，輸出結果卻不一致等問題。為了解決這些問題，我們設計一個以 Agent 為核心的流程架構，負責接收 SLM 分別生成的工具名稱與參數，組成完整的工具調用（Tool Call），並據此執行對應的視覺化工具操作，包括前端圖表更新、後端資料查詢，以及處理輸入中的資訊缺漏與非視覺化相關請求，進一步提升互動流程的彈性與穩定性。

在實驗中，我們採用公開的 ChartGPT 資料集進行驗證，並以常用於機器翻譯評估的兩項指標——ROUGE-L 與 BLEU——來衡量參數生成的準確性。實驗結果顯示，微調後的 Phi-3.5 模型在參數生成準確性上有明顯提升，其中 ROUGE-L 較目前以小型語言模型為基礎的視覺化系統 ChartGPT 提高約 1%，而 BLEU 表現則相當。進一步與三個未經視覺化任務微調的一般化語言模型（xLAM、LLaMA3.2 與 Qwen2.5）相比，我們的模型在 ROUGE-L 指標上平均提升約 78%，在 BLEU 指標上則平均提升約 48%。此外，在使用私有的 6G 資料集進行測試時，即使面對使用者輸入不完整的情境，系統仍能生成符合需求的視覺化圖表，展現出良好的強健性（robustness）。

關鍵字：視化導向自然語言互動介面，大型語言模型，小型語言模型，代理，工具調用

Abstract

Traditional visualization-oriented natural language interfaces (V-NLIs) primarily rely on drop-down menus to generate charts, which are less intuitive than natural language input. As a result, recent research has increasingly incorporated large language models (LLMs) to generate visualizations through natural language commands. While LLMs have demonstrated strong performance in visualization, their growing hardware and GPU resource demands have made local deployment challenging. The situation has led to the adoption of lightweight small language models (SLMs) as an alternative solution.

Although SLMs offer the advantage of lightweight deployment, they still face several challenges in generating visualizations—such as producing chart parameters that do not meet user expectations or yielding inconsistent outputs for the same input. To address these issues, we propose a framework centered around an agent, which receives the tool name and parameters separately generated by the SLM, assembles them into a complete tool call format (also known as a function call), and executes the corresponding visualization tool operations. These operations include frontend chart updates, backend data queries, and handling missing input information or non-visualization-related requests, thereby enhancing the flexibility and stability of the overall interaction process.

In our experiments, we used the publicly available ChartGPT dataset for evaluation. We adopted two commonly used metrics in machine translation—ROUGE-L and BLEU—to assess the accuracy of parameter generation. The results show that the fine-tuned Phi-3.5 model significantly improves parameter generation accuracy. Specifically, it outperforms the existing visualization system ChartGPT based on a small language model by approximately 1% in ROUGE-L, with comparable performance in BLEU. Furthermore, compared to three general-purpose language models that were not fine-tuned for visualization tasks (xLAM, LLaMA3.2, and Qwen2.5), our model achieves an average improvement of approximately 78% in ROUGE-L and 48% in BLEU. In addition, tests on a private 6G dataset demonstrate that the system

can still generate appropriate visualization charts even when faced with incomplete user input, showcasing strong robustness.

Keywords: visualization-oriented natural language interfaces , large language model , small language model , agent , tool call

誌謝

首先，誠摯感謝我的指導教授林家瑜老師。在毫無相關作品背景、從資管系跨考資工的起點上，老師仍願意給予我機會並接納我，讓我有機會踏上研究這條路。在整個研究歷程中，老師安排我參與 6G 計畫，並指派我管理實驗室設備，使我累積了寶貴的實作經驗與研究能力。論文撰寫期間，老師亦耐心協助我釐清研究問題與方向，並適時提醒我強調在論文中的個人貢獻，使本論文得以聚焦與成形。

接著感謝 6G 計畫的郭志義老師。在每日晨會中，老師總是協助我們解決前一日所遇到的問題，並透過每週的論文報告持續鍛鍊我們的簡報與表達能力，讓我學會更有效地閱讀論文並從同儕的報告中吸收經驗。此外，郭老師也在語言模型相關領域提供了我許多建設性的建議，使本研究得以順利完成。

也感謝實驗室的學長姊、同儕與學弟妹們，無論是在課業上的協助，還是在課餘時光中的陪伴與娛樂，每一次的討論、聚餐與閒聊，都是我能夠持續撰寫論文的重要動力。此外，特別感謝一位大學時期的朋友，總是在我壓力繁重、情緒低落時主動邀約我散心、傾聽我的煩惱，讓我得以適時釋放壓力、重新調整步伐。

最深的感謝，留給我的家人。謝謝你們始終如一地支持我完成碩士學業，尤其在我長時間待在家中、研究進度緩慢的那段日子裡，仍讓我無後顧之憂地生活，並從未對我有絲毫責備。正因為有你們的理解與包容，我才能安下心來完成這份研究，走完這段求學旅程。

目錄

	頁次
摘要	i
Abstract	ii
誌謝	iv
目錄	v
圖目錄	vii
表目錄	viii
一、 緒論	1
1.1 研究背景	1
1.2 研究動機	1
1.3 問題定義	2
1.3.1 SLM 難以生成 Tool Call 所需的關鍵資訊	2
1.3.2 ChartGPT 流程缺乏彈性	3
1.3.3 Tool Call 在視覺化任務上的應用不足	4
1.4 研究目標	4
1.5 研究貢獻	5
二、 相關研究	6
2.1 LLM 在 V-NLI 系統中的整合	6
2.2 SLM 在 V-NLI 系統中的整合	7
2.3 強化 Tool Call 能力的語言模型	7
2.4 相關研究的比較	8

三、	方法	9
3.1	SLM 微調策略與流程設計	10
3.1.1	資料集介紹	10
3.1.2	模型選擇	11
3.1.3	微調流程與訓練設定	12
3.2	視覺化工具設計	13
3.3	Agent 運作流程設計	14
3.3.1	輸入驗證階段	15
3.3.2	Tool Call 生成階段	15
3.3.3	指令執行階段	16
四、	實驗	18
4.1	資料集	18
4.2	評估指標	19
4.2.1	ROUGE-L	19
4.2.2	BLEU	20
4.3	實驗一	21
4.3.1	實驗流程	21
4.3.2	結果分析	22
4.4	實驗二	23
4.4.1	使用場景介紹	23
4.4.2	實驗流程	24
4.4.3	實驗結果比較	24
4.4.4	實驗二討論	27
五、	總結	29
5.1	結論	29
5.2	未來展望	29
六、	參考文獻	31

圖目錄

	頁次
1.1 SLM Tool Call 輸出錯誤範例	3
3.1 系統架構圖	9
3.2 方法流程圖	10
3.3 SLM 微調流程圖	13
3.4 Agent 驗證流程圖	15
3.5 Tool Call 生成流程圖	16
3.6 Agent 執行工具流程圖	16
3.7 Grafana 互動介面展示	17
4.1 prometheus 資料庫預覽	19
4.2 Level 1 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法	25
4.3 Level 2 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法	26
4.4 Level 3 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法	26
4.5 Level 4 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法	27

表目錄

	頁次
2.1 V-NLI 系統與 Tool Call 模型之比較	8
3.1 ChartGPT 資料集範例	11
4.1 資料集所使用到的單一 CSV 檔範例內容	19
4.2 不同模型於 ChartGPT 測試集上之參數生成表現	23
4.3 Robustness 測試中四種輸入資訊完整度場景	23

一、緒論

1.1 研究背景

在大數據時代，隨著每日由電腦系統所產生的資料量持續攀升，直接閱讀或分析未經處理的原始資料已變得日益困難，不僅耗時且難以掌握其中的關鍵資訊。為因應此挑戰，資料視覺化逐漸受到重視，透過將龐大的資料轉化為易於理解的圖表，能夠協助使用者快速辨識資料中的異常現象與趨勢。

常見的視覺化平台如 Grafana，支援多種資料庫整合與多樣化圖表呈現，並透過色彩與圖例強化資料的可讀性。儘管 Grafana 視覺化功能完整，但其圖表的建立仍然依賴傳統的下拉式選單操作，對於初次使用該介面的使用者而言，往往需耗費大量時間進行嘗試與設定。隨著大型語言模型（Large Language Models, LLMs [1]）的快速發展，自然語言輸入成為人機互動的重要方式，相較之下，傳統表單操作已顯得不夠直觀。

為因應上述使用者互動習慣的改變，研究人員開始將自然語言處理技術應用於視覺化任務，進而提出可視化導向自然語言互動介面（Visual Natural Language Interface, V-NLI [2]）系統。V-NLI 系統允許使用者以自然語言描述視覺化需求，並自動解析輸入內容以產生對應圖表，進一步降低操作門檻，提升互動的直覺性與效率。

1.2 研究動機

目前已有研究透過自然語言輸入來生成視覺化圖表，例如 ChartGPT [3]，其設計將整體圖表的生成任務拆解為六個連續的子任務，分別為：欄位選擇、資料篩選、欄位聚合操作、圖表類型決定、視覺編碼調整（如 XY 軸欄位對應），以及是否對 XY 軸的資料進行排序。系統透過逐步推理的方式，依序完成每個子任務，最終將所有步驟整理成合法的 Vega-Lite 語法，再交由視覺化平台呈現最終圖形。

儘管 ChartGPT 的設計驗證了透過自然語言生成視覺化圖表的可行性，但其流程相對固定，對輸入資訊的完整性要求較高。當使用者輸入的敘述過於模糊或遺漏關鍵條

件時，系統無法有效補全缺失資訊，導致圖表生成失敗或結果與預期不符。為了改善上述限制，我們決定引入工具調用（Tool Call [4]），將每一項視覺化子任務視為可獨立調用的工具，讓系統能根據使用者輸入動態決定需執行哪些工具，進而實現更具彈性的流程。

然而，要正確生成結構完整的 Tool Call 仍具挑戰性。過去多數研究仰賴 LLM 來處理此類任務，但 LLM 的訓練成本與運算資源需求極高，不僅難以在本地端或計算資源有限的設備上部署，也常需透過雲端服務運行，對於企業來說可能存在資料隱私與資安風險。因此，我們進一步探索小型語言模型（Small Language Model, SLM [5]）的應用潛力。SLM 相較於 LLM 擁有更低的硬體需求與更快的回應速度，不僅適合部署於手機、穿戴式設備等邊緣裝置，也更利於在開發階段或資料敏感環境中進行本地部署，避免資料外洩風險。在成本、效能與資安的多重考量下，SLM 成為了本研究用以生成 Tool Call 的基礎。

1.3 問題定義

1.3.1 SLM 難以生成 Tool Call 所需的關鍵資訊

SLM 雖具備輕量化的優勢，但在生成 Tool Call 格式時仍存在多項挑戰，如圖 1.1 所示。在未經微調的情況下，常見以下三類問題：

（一）**格式錯誤**：Tool Call 格式中僅應包含工具名稱及其對應參數，任何額外註解皆屬冗餘。此外，所有參數名稱與對應內容皆須正確置於 parameters 欄位中，若模型產出超出格式規範的語句或將參數置於錯誤位置，將可能導致系統無法正確解析。

（二）**參數內容錯誤**：模型所產生的 Tool Call 參數內容應符合使用者的預期。例如，當使用者明確指定繪製折線圖時，若產出為長條圖，即為錯誤結果。此外，在視覺編碼方面，常見的時間欄位通常對應至 X 軸，若模型將其配置於 Y 軸，也會造成結果與一般認知不符。

（三）**一致性不足**：即使透過調整模型的超參數（如 temperature）以降低輸出的隨機性，SLM 仍難以保證在相同輸入下產生一致的結果。此不穩定性在實際應用中不僅可能造成使用者混淆，也會降低對系統輸出的信任，進而影響整體操作體驗。

```

{
  "name": "create_chart",
  "description": "Create a Vega Lite style chart template to visualize `tx_brake_downlink` for all timestamps.",
  "parameters": {
    "columns": ["Timestamp"],
    "chart_type": "line" // Assuming line plot is suitable; if not, specify another type like bar or scatter.
  },
  "encodings": [
    { "field": "Timestamp", "type": "temporal" },
    { "field": "tx_brake_downlink", "type": "quantitative" }
  ]
}

```

圖 1.1: SLM 輸出之 Tool Call 錯誤範例，對應上述第（一）與（二）點。該輸出包含多餘的 description 欄位與註解，encodings 欄位未正確置於 parameters 結構中，且缺少視覺通道欄位（如 X 或 Y 軸）的對應設定，顯示其在格式結構與參數內容上皆存在明顯錯誤。

1.3.2 ChartGPT 流程缺乏彈性

此限制於 ChartGPT [3] 原文中提及。作者指出，由於系統視覺化能力有限，當使用者輸入的意圖超出支援範圍時，系統仍然能夠產出結果，但實際上與使用者需求不符，且缺乏明確提示機制，讓使用者難以理解其限制所在。本節將以公式化方式定義 Tool Call 流程，並進一步說明固定流程與其所帶來的兩項問題。

$$\{T_1, \dots, T_m\} = f(Q, \{D_1, \dots, D_n\}) \quad (1.1)$$

其中：

- Q 表示使用者輸入的自然語言敘述。
- $D_1 \sim D_n$ 是使用者提供的資料內容，最多包含四項：
 - D_1 ：欄位名稱的集合
 - D_2 ：欄位資料型態的集合
 - D_3 ：資料表名稱
 - D_4 ：實際資料內容（或範例值）的集合
- $T_1 \sim T_m$ 表示生成出的 Tool Call 集合，每一個 T 都包含以下內容：
 - 工具名稱 N
 - 該工具所需的固定參數 $\{P_1, \dots, P_p\}$

以 ChartGPT 為例，其在公式中的輸入與輸出結構皆為固定，其中 $n = 4$ 、 $m = 6$ 。然而，這樣缺乏彈性的設計存在兩項問題，分別為：

(一) **缺乏補全與回饋機制以處理例外情況**：當使用者提供的資料不夠完整時（即輸入的資料數 $n < 4$ ），系統應具備補全或推測缺漏資訊的能力，或在使用者需求超出系統支援範圍時，能即時回饋並提示限制所在。若系統僅依賴固定流程進行判斷與執行，將難以應對資料不全或需求異常等例外情況。

(二) **不必要操作仍被固定執行**：並非所有工具在每次視覺化任務中皆為必要（所需工具數 $m < 6$ ）。在 ChartGPT 的流程中，聚合操作與排序操作為固定步驟，但若使用者僅希望簡單呈現某一指標（如以長條圖顯示單一數值欄位），這些額外操作可能反而無法呈現原始資料樣貌。

在上述兩種情況下，我們認為系統中應引入一個代理（Agent），以處理資料不足與彈性選擇工具的問題。此 Agent 除了須具備判斷能力，也要能正確解析由 SLM 所產生的 Tool Call 格式，並實際執行相關的工具操作，以實現視覺化流程的自動化。

1.3.3 Tool Call 在視覺化任務上的應用不足

為了完成 Tool Call 的後續流程，系統在 Agent 解析出所需工具後，必須具備對應的工具以執行其內容。特別是在實現圖表生成自動化的情境下，繪圖工具是不可或缺的組件；若使用者的資料儲存於資料庫中，亦需搭配資料擷取工具，以支援資料存取與查詢操作。

然而，現有的工具設計多以一般任務為導向 [6]，較少針對視覺化需求進行優化或專門開發。因此，若欲實現視覺化流程的完整自動化，對應的視覺化工具開發將是不可或缺的一環。

1.4 研究目標

為了解決上述三項問題，我們提出一套由 SLM 驅動的 Agent 運作流程，作為 V-NLI 系統的核心設計，並以 Grafana 為例進行實作與驗證來達成以下目標：

(一) **微調 SLM 以提升 Tool Call 的參數生成能力**：我們微調了 Phi-3.5 [7] 作為本研究所使用的 SLM，以提升其在 Tool Call 任務中的參數生成準確性，並設計實驗驗證其成效。

(二) **設計具決策與修正機制的 Agent**：我們設計了一個具備決策與錯誤檢查能力的 Agent，能夠處理輸入資料不完整的情況，確保所生成的 Tool Call 格式正確，並執行

相對應的工具內容，以完成視覺化任務。

(三) **設計以視覺化任務為導向的工具**：我們開發了一套專為視覺化任務設計的工具，只要模型能正確生成符合格式的 Tool Call，並有對應的角色執行其內容，即可實現在 Grafana 上自動化產生與更新圖表的功能。

我們共進行兩組實驗，以驗證本系統的可行性與效能。第一組實驗使用 ChartGPT 公開資料集，透過 ROUGE-L 與 BLEU 指標，評估微調後模型在參數生成方面的準確性，並與未經視覺化任務微調的通用語言模型（如 xLAM、LLaMA3.2 與 Qwen2.5）及現有系統 ChartGPT 進行比較。第二組實驗則採用私有的 6G 專案資料，測試系統在使用者輸入資料不完整情況下的強健性。綜合實驗結果顯示，系統不僅能產出準確且可執行的 Tool Call，亦能完成視覺化任務的自動化流程。

1.5 研究貢獻

本研究的貢獻可歸納為以下三點：

(一) 我們微調後的 Phi-3.5 模型在視覺化參數生成任務中表現優於現有系統 ChartGPT，在公開資料集上 Rouge-L 提高約 1%，BLEU 則與其相當。進一步與三個未經視覺化任務微調的一般化語言模型（xLAM、LLaMA3.2 與 Qwen2.5）相比，我們的模型在 ROUGE-L 指標上平均提升約 78%，在 BLEU 指標上則平均提升約 48%。

(二) 系統在私有資料測試中展現出良好的 Robustness，即使面對輸入資訊不完整的情況，仍能正確生成符合需求的視覺化圖表。

(三) 提出一套以 SLM 負責生成、Agent 負責決策與執行的 Tool Call 流程，具備平台無關性（platform-agnostic），並成功實作於 Grafana，未來可擴展至其他視覺化平台。

二、 相關研究

2.1 LLM 在 V-NLI 系統中的整合

在早期的使用者介面中，視覺化圖表的產出往往依賴視窗（Windows）、圖示（Icons）、選單（Menus）與指標（Pointer）這四項核心元件，簡稱為 WIMP [8]。隨著 LLM 的發展，逐漸出現以自然語言作為輸入手段的視覺化系統。使用者期望能透過簡單的語句生成複雜圖表，不僅可降低操作負擔，也有助於快速掌握系統的圖表生成能力。

Chat2VIS [9] 是早期將 LLM 應用於視覺化生成任務的代表性系統之一。該系統透過提示工程（prompt engineering [10]），將使用者輸入與資料欄位資訊轉換為語言模型容易理解的 prompt，用於說明資料內容與請求生成繪圖程式碼，並透過 GPT-3 產出對應的 Python 程式碼以生成圖表。然而，當時模型的推理能力有限，系統在正確性與一致性上皆存在問題。即使輸入相同的敘述，模型仍可能產出不同版本的程式碼，導致前後輸出的圖形不一致，以至於混淆使用者。

為了提升 LLM 在圖形生成上的準確度，有研究在流程中加入人工調整的機制，例如 DynaVis [11]。該系統會根據使用者輸入，透過 GPT-3.5 產生對應的 Vega-Lite 規格並顯示初始圖表。若使用者希望進一步修改圖形，系統會交由 LLM 產出符合目前模板的 HTML 與 JavaScript 程式碼，在前端動態生成控制視窗協助使用者進行微調。這種方法雖能滿足使用者需求，但過多的互動元件也可能造成介面複雜，增加系統負擔，甚至導致使用者因彈出視窗頻繁而產生混淆。

近兩年的研究則是陸續導入代理（Agent）協助處理資料分析與圖表生成任務。其中，MatplotAgent [12] 提出一種由三個 Agent 組成的架構，包含 Query Expansion、Code Agent 與 Visual Agent，不僅支援程式碼的自我修正（self-debugging），也能透過多模態模型觀察圖像結果並提出修改建議，有效提升圖表品質。另一方面，nvAgent [13] 則設計 Processor、Composer 與 Validator 等三個 Agent，分別負責資料篩選、視覺化規格生成與執行驗證流程，確保結果在邏輯與語法上皆正確。儘管 Agent 能有效輔助 LLM 修

正錯誤，但隨著模型複雜度的提升，系統對 Agent 的依賴程度逐漸降低。此外，現有系統多採用 GPT-4 等大型模型，較少考量實際部署成本與運算資源限制。

2.2 SLM 在 V-NLI 系統中的整合

儘管現有系統透過 LLM 已能實現高度自動化的圖表生成流程，但其龐大的模型參數與計算資源需求，對於本地部署而言仍是重大挑戰。因此本研究進一步探討 SLM，根據 Zhao 等人於文獻 [1] 中的說明，LLM 通常指參數量超過 10 億的模型。據此，本研究將 SLM 定義為參數量不超過 10 億的語言模型。

目前將 SLM 應用於視覺化任務的研究仍相對有限。除了 ChartGPT [3] 採用將任務拆解為多個步驟以強化推理能力之外，V-RECS [14] 亦提出一套以 LLaMA-2-7B 為基礎的 SLM 系統，並設計教師-學生訓練流程，透過 GPT-4 產出包含解釋（Explanation）、說明（Captioning）與延伸建議（Suggestions）等敘述，進一步用以微調 SLM，使其能同時生成圖表與相關文字說明。儘管 V-RECS 強調圖表與說明內容的搭配性，其實驗僅與 GPT-4 進行比較，缺乏與其他系統在圖表品質上的對照，且所生成文字的正確性與實用性亦未提供明確的量化驗證。

2.3 強化 Tool Call 能力的語言模型

除了在流程中加入 Agent，陸續有研究探索語言模型執行 Tool Call（又稱 Function Calling）流程的能力，並設計流程來專門生成訓練資料集。

xLAM [15] 整合了多個資料集（如 ToolBench、BFCL 等），對各個來源的資料進行了格式統一與錯誤分類，並依據指令類型（Function Calling、Instruction Following、Reasoning 等）進行資料增強與混合，最終以監督式微調（Supervised Fine-Tuning, SFT [16]）與直接偏好最佳化（Direct Preference Optimization, DPO [17]）進行模型訓練，強化模型在不同任務場景下的能力。

ToolACE [18] 則進一步針對 Tool Call 的資料品質進行控管，提出 MAI（Multi-Agent Interactive）資料生成流程，模擬使用者、Agent 與工具之間的多輪互動。該流程透過多次生成與多數決策策略來檢查工具調用的一致性，並在資料中加上 Loss Mask 控制訓練信號，以避免模型學習到錯誤樣本。

儘管上述研究在 Tool Call 資料設計與模型訓練方面提供了完善的處理流程，然而其應用場景大多聚焦於較為一般化的 API 操作（如查詢商品、操作日曆、撰寫郵件等），在 API 相關的工具組成中並未特別包含資料視覺化任務。

2.4 相關研究的比較

表 2.1 比較了各 V-NLI 與 Tool Call 系統在多個層面的設計。首先，我們整理各系統所採用的語言模型，並依據是否符合本研究對 SLM 的定義進行判斷。接著，參考 Huang 等人於文獻 [19] 中提出的 FT (Fine-tuning) 指標，用以評估各系統是否針對視覺化任務進行模型微調。最後，我們納入強健性 (Robustness) 作為比較指標。儘管目前尚無研究廣泛將此指標用於 V-NLI 系統的比較，然而在引用次數較高的文獻 [20] 中已明確指出，Robustness 將在 V-NLI 系統中扮演日益關鍵的角色，特別是在使用者輸入模糊或資料不完整的情況下，系統的 Robustness 將直接影響圖表生成的準確性與成功率。因此，在本研究中，我們以各系統是否於流程中設計處理資訊不足、模糊敘述或例外情境的機制，作為其 Robustness 特性的判斷依據。

Robustness 具體的判斷方式，例如：DynaVis [11] 透過動態元件提供使用者即時調整能力，即使初始指令不完整也能透過互動補足需求；nvAgent [13] 與 MatplotAgent [12] 則設計了 Validator 或 Visual Agent 等模組，能於生成錯誤時自動偵測並進行修正；XLAM [15] 與 ToolACE [18] 則採用了語意檢查器與驗證流程，確保所產生的 Tool Call 合法且符合執行條件。這些設計皆有助於系統在面對不理想輸入時維持穩定表現，因而被本研究視為具備 Robustness 特性。

表 2.1: V-NLI 系統與 Tool Call 模型之比較

系統名稱	Base Model	SLM	FT	Robustness
Chat2VIS [9]	GPT-3	✗	✗	✗
DynaVis [11]	GPT-3.5-turbo	✗	✗	✓
MatplotAgent [12]	GPT-4,GPT-4V	✗	✗	✓
nvAgent [13]	GPT-4o	✗	✗	✓
V-RECS [14]	Llama-2	✓	✓	✗
ChartGPT [3]	Flan-T5-XL	✓	✓	✗
xLAM [15]	DeepSeek-Coder	✓	✗	✓
ToolACE [18]	Llama-3.1	✓	✗	✓
Ours	Phi-3.5	✓	✓	✓

三、 方法

為了將 Tool Call 應用於視覺化任務，並解決 SLM 在生成過程中常見的格式錯誤與參數不準確問題，本研究提出一套以 Agent 為核心的系統架構，如圖 3.1 所示。其中，圖中以紅色方框標示的元件為本研究所開發，至於 SLM 部分則使用 Microsoft 所推出的 Phi-3.5 模型，並以 ChartGPT 提供的資料集進行微調，因而未特別框出。整體架構包含三個主要角色：SLM、Tool 與 Agent，分別負責語意理解與參數生成、視覺化功能執行，以及流程控制與工具調用，協同完成自然語言至圖表生成的自動化流程。

詳細流程如圖 3.2 所示，由 Agent 主導執行。流程首先由 Agent 進行輸入驗證，針對使用者輸入進行任務判斷與資料補足；接著由 SLM 根據輸入與工具列表，逐步生成候選的工具名稱與對應參數；最後，Agent 組合所有資訊為合法的 Tool Call 並執行相應工具，完成圖表生成與資料查詢等任務。最終的視覺化結果將呈現於 Grafana 平台上。

接下來的章節將依序介紹系統中三個角色的設計與實作：首先，第一節將介紹本研究所使用的 SLM 及其微調流程；第二節介紹我們為了視覺化任務所設計的工具；第三節則介紹 Agent 的決策流程，並說明其如何透過驗證、生成與執行三個階段，實現視覺化圖形生成的自動化。

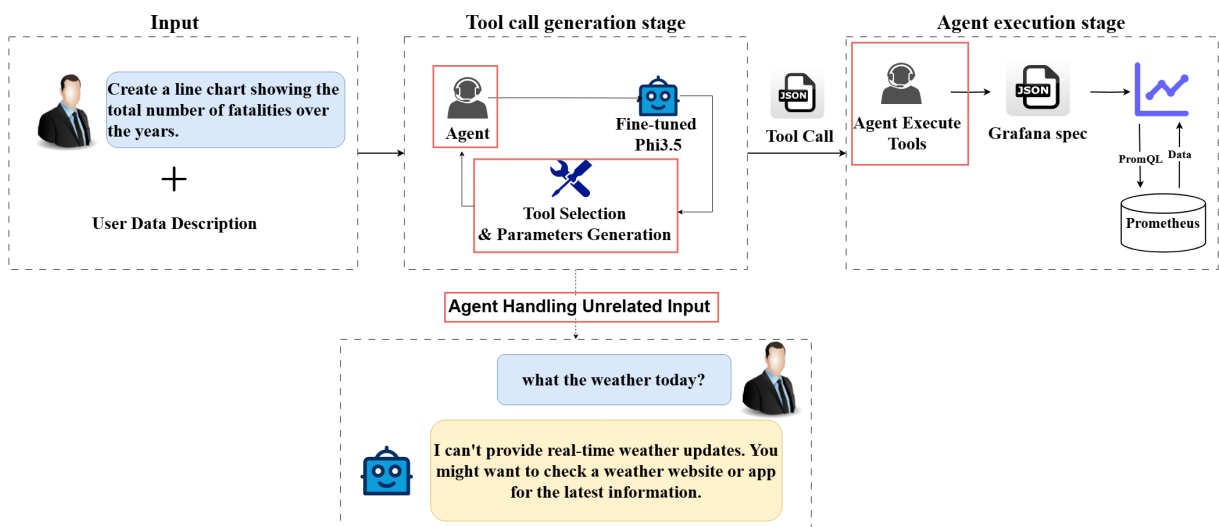


圖 3.1: 系統架構圖

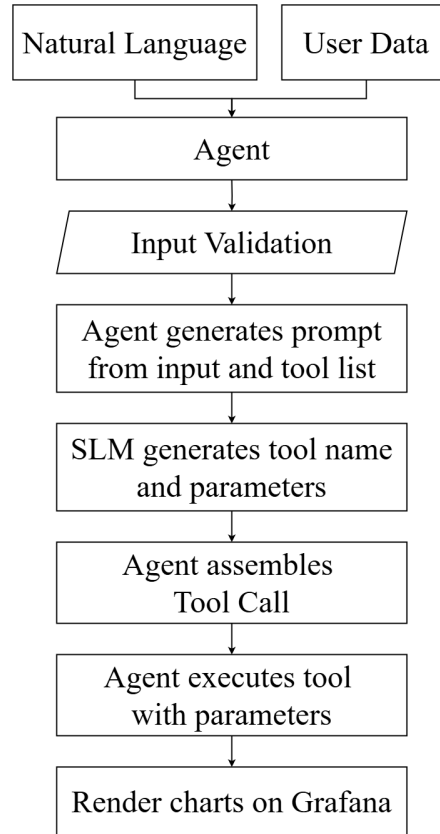


圖 3.2: 方法流程圖

3.1 SLM 微調策略與流程設計

由於目前公開的 Tool Call 訓練資料集中，大多缺乏針對視覺化任務設計的資料，例如本研究早期曾採用 xLAM [15] 通用資料集進行訓練，雖然能使 SLM 學會生成符合格式的 Tool Call，卻在視覺化任務中面臨參數準確率不佳的問題。

為了解決此限制，本研究改採 ChartGPT [3] 所提供的視覺化資料集進行微調。該資料集的優點在於針對視覺化任務中六個常用參數進行逐步訓練，且在輸入資料中，包含了欄位名稱集合與資料範例等資料，有助於提升模型在參數生成上的準確度。然而，由於該資料集並未涵蓋完整的 Tool Call 結構，因此本節聚焦於提升 SLM 在單一視覺化參數上的生成準確率。

3.1.1 資料集介紹

ChartGPT [3] 所提供的資料集共有超過 10,000 筆的視覺化資料，實際上由約 1,900 筆不同任務擴充而成。每個任務皆包含六個視覺化生成步驟，這些步驟涵蓋圖表類型選擇與資料處理需求，其中欄位選擇通常從使用者提供的資料欄位中挑選，其他步驟

的參數則包括：圖表類型涵蓋長條圖、折線圖、圓餅圖與散點圖；篩選條件常見運算子包含 $>$ 、 $<$ 、 $=$ 、 $\neq$ 、 $\geq$ 、 $\leq$ ；聚合方式包含加總，計數，取平均等常見操作；視覺通道的編碼用於指定欄位與 X、Y 軸之間的對應關係；排序則用於定義在 X 或 Y 軸上的資料如何排序，例如 x asc 或 y desc。

每筆訓練資料均包含 prompt 與 completion 欄位如表 3.1 所示，其中 prompt 為模型輸入，整合了資料表名稱、欄位名稱集合、欄位資料型態集合、使用者查詢語句、當前子任務的提示詞，以及前一步驟的輸出作為上下文；completion 則為對應子任務的正確參數輸出。

表 3.1: ChartGPT 資料集範例

prompt	completion
Faculty <head> FacID, Lname, Fname, Rank, Sex, Phone, Room, Building <type> quantitative, nominal, nominal, nominal, nominal, quantitative, nominal, nominal <data> 1082, Giuliano, Mark, Instructor, M, 2424, 224, NEB <line> 1121, Goodrich, Michael, Professor, M, 3593, 219, NEB <line> <utterance> What does the composition of faculty members look like based on their roles? <ans> <sep> Step 1. Select the columns:	Rank

3.1.2 模型選擇

在本研究中，我們選擇 Microsoft 所開源之 Phi-3.5 作為基礎模型，選擇依據主要有兩點，相關數據均來自官方技術報告文件 [21]：

(一) Phi-3.5 展現出優異的推理能力 (reasoning)。以 ARC Challenge (10-shot) 為例，Phi-3.5-MoE 可達 91.0 分，優於 Llama-3.1-In 的 83.1 分、Gemma-2 的 89.8 分與 Mixtral-8x7B 的 84.8 分。由於本研究所處理的任務主要聚焦於 Tool Call 的參數生成與資料補全，推理能力將影響模型在資訊不足情況下的正確生成表現，故為本研究評估模型時之關鍵考量。

(二) Phi-3.5-mini 採用僅 3.8B 的輕量設計，於 MMLU (5-shot) 基準測試中仍可達 69% 準確率，與 Llama-3.1-In 8B 的 68.1% 及 Gemma-2 9B 的 71.3% 相當。MMLU [22] 所涵蓋之題型多為跨領域綜合性問題，包含常識、邏輯、數學與歷史等範疇，需結合知識與條件進行合理判斷，屬於典型的推理任務，故可作為衡量模型推理能力的重要

指標之一。

考量到推理能力與部署成本的綜合性價比，本研究最終選擇了 Phi-3.5-mini 作為整個架構的基礎。

3.1.3 微調流程與訓練設定

參照 ChartGPT 所定義的六個視覺化步驟，其訓練流程如圖 3.3 所示。微調過程中，我們依序針對六個視覺化參數進行訓練，每個步驟皆對應一個簡短的提示詞，分別為：

- Step 1: Select columns:
- Step 2: Add filter:
- Step 3: Add aggregations:
- Step 4: Select chart type:
- Step 5: Choose encoding:
- Step 6: Add sort:

在模型選擇方面，如第 3.1.2 節所述，我們採用 Phi-3.5-mini 作為本研究所使用之 SLM 基礎架構。微調過程採用 SFT [16] 方法，主要超參數設定如下：

- Global batch size : 16
- Epoch : 3
- Learning rate : 1e-4
- Optimizer : AdamW
- Warm-up steps : 50

透過上述操作，我們訓練出一個能夠接收自然語言輸入，並準確產出視覺化參數的輕量模型，作為後續 Tool Call 中參數生成的基礎。

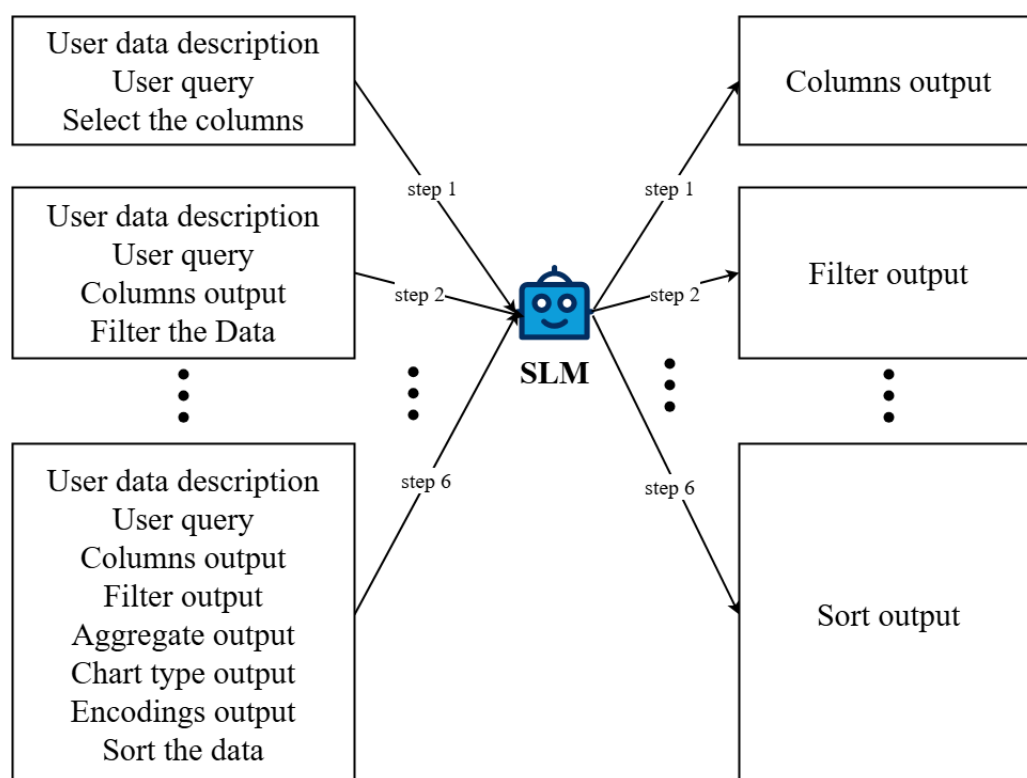


圖 3.3: SLM 微調流程圖

3.2 視覺化工具設計

在完成 SLM 微調後，雖然模型已具備準確產出六個視覺化參數的能力，但在實際應用上，僅依賴這些參數仍不足以自動化更新圖表至視覺化平台。因此，我們進一步設計了一組可被 Agent 調用的視覺化工具，以對應不同任務的實際需求。

本研究最終設計五個工具，分別對應圖表建立、資料轉換、變數篩選、SQL 查詢與圖表上傳等任務。其中圖表建立、SQL 查詢與圖表上傳為每次任務中皆會使用的核心工具，且具明確的執行順序，系統會先完成圖表與資料設定，待視覺化內容準備完成後，再透過圖表上傳工具將結果呈現至 Grafana 平台。

（一）**圖表建立工具**（create_chart）此工具為圖表生成流程中最基本的元件，其所需的參數包含欄位名稱、視覺編碼以及圖形類別，皆由 SLM 所產出。這三項資訊將會被轉換為 Grafana 所接受的視覺化規格，進一步解析視覺編碼結果，以決定 X 與 Y 軸的欄位對應關係，並將圖形類別轉換為 Grafana 所使用的圖表設定名稱（如 bar 對應為 barchart 等）。整體設計目的在於簡化圖表建立流程，使系統能夠順利地產出基本圖形。

（二）**資料轉換工具**（apply_transformation）此工具負責處理資料的轉換需求，當使用者查詢中涉及欄位彙整或資料排序時，系統將呼叫此工具進行對應處理。其所需的參數由 SLM 所產出，包含欄位聚合與排序設定，並非於每次任務中皆為必要。

此工具對應於 Grafana 中的 Transform 功能，主要操作包括群組（Group by）與排序（Sort by），並可結合常見的彙整函數（如加總、計數、平均等）進行資料彙整。

（三）**變數篩選工具**（set_variable）此工具用於設定 Grafana 儀表板中的變數條件，以達到資料篩選的目的。其所需參數由 SLM 產出，主要為欄位名稱與對應的篩選條件。

此工具對應至 Grafana 的變數控制機制，可在不修改查詢語句的情況下，動態變更圖表所呈現的資料範圍，常用於指定時間範圍或特定欄位值的切換。

（四）**SQL 查詢工具**（generate_sql）此工具負責根據使用者查詢意圖，自動產生資料查詢語句，以支援 Grafana 從資料庫中擷取所需資料。工具所需的參數包含欄位選擇、篩選條件與聚合設定，由 SLM 提供並組合成合法查詢語句。在本研究中，使用的資料庫為 Prometheus，因此查詢語言主要為 Prometheus 的 PromQL(Prometheus Query Language) 格式。

PromQL 語句的組成通常包含一個欄位名稱、可選的標籤過濾條件，以及視需求加入的聚合運算與時間視窗。例如：

```
sum(rate(tx_brake_downlinkslice="1"[5m]))
```

代表對指定 slice 的下行傳輸速率，在過去 5 分鐘內取平均變化率，並加總所有實體設備的結果。

（五）**圖表上傳工具**（upload_chart）此工具負責完成整個視覺化流程的最後一個步驟：將生成完成的圖表內容上傳至指定的 Grafana。當前述各項工具完成參數設定、資料查詢與圖表樣式轉換後，本工具會整合所有輸出結果，轉換為合法的 Grafana 規格，並透過 API 傳送設定至 Grafana，完成圖表的建立與呈現。

由於本研究採用聊天機器人作為前端自然語言輸入介面，並將其嵌入至特定 Grafana 儀表板中，因此所有圖表皆會被動態地新增至該儀表板中統一顯示，確保整體操作流程的一致性。

3.3 Agent 運作流程設計

為整合 SLM 微調結果與前述各項視覺化工具的操作流程，本研究設計了一個 Agent，負責處理從使用者輸入到工具執行的整體流程。整體運作流程可劃分為三個階段：輸入驗證階段、Tool Call 生成階段，以及工具執行階段。

現有研究常將語言模型視為 Agent 的一部分，使其僅服務於特定任務（如 Tool Call 生成）。然而，本研究所設計之 Agent 架構則採用不同策略：SLM 作為獨立元件存在，

由 Agent 中的各個功能單元（如輸入驗證與 Tool Call 生成）主動向 SLM 發送查詢，以推測缺失的使用者資料或者進行參數的生成。如此一來，SLM 並非綁定於單一處理階段，而是作為通用處理引擎，支援各階段任務，進而提升整體架構之模組彈性與可維護性。

3.3.1 輸入驗證階段

如圖 3.4 所示，在此階段，Agent 首先判斷使用者查詢是否與視覺化任務相關，並確認是否已提供足夠的資料以進行圖表生成。若查詢無效，系統會直接回傳錯誤訊息；若資料不完整，Agent 則會根據輸入語句，試圖補齊缺失資訊以利後續處理。

其中，欄位選擇為視覺化任務中最關鍵的參數之一，若使用者未提供任何欄位名稱集合在其資料說明裡，系統將無法正確查詢與繪圖。因此，當 Agent 偵測到缺少欄位資訊時，會依據使用者輸入內容，自動產生提示詞，引導 SLM 從原始語句中推論出可能的欄位名稱。

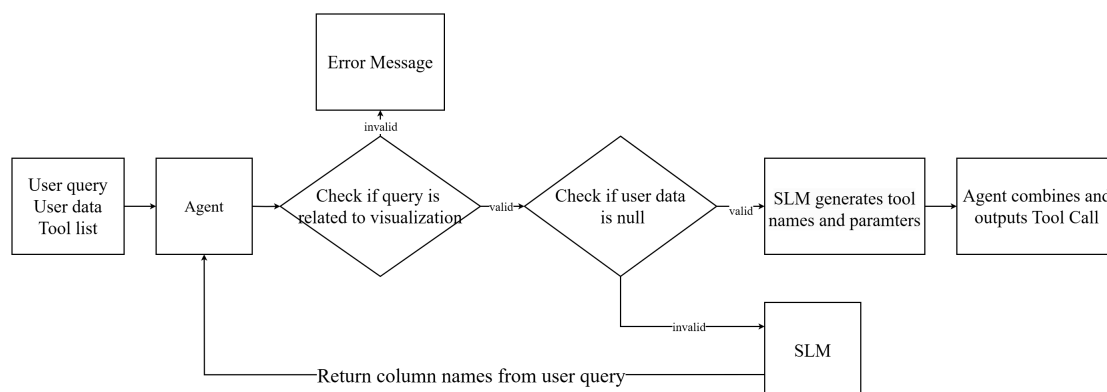


圖 3.4: Agent 驗證流程圖

3.3.2 Tool Call 生成階段

由於前述微調階段僅針對單一視覺化參數進行訓練，並未涵蓋完整 Tool Call 所需的格式，因此本研究額外設計 Agent 與 SLM 搭配運作的流程，用以補足 Tool Call 的完整性。儘管未針對工具選擇進行微調，但由於可用工具數量有限，加上 Phi-3.5 模型本身具備基本的語意理解能力，因此在實作中，我們採用 Agent 引導 SLM 逐步確認任務所需的工具與參數，藉此構成合法且可執行的 Tool Call。

如圖 3.5 所示，Agent 會首先詢問工具名稱（如 `create_chart`），再依序詢問欄位名稱、圖表類型與視覺編碼等資訊。透過逐步確認與記憶每一步驟的輸出，Agent 能有效

追蹤當前任務的進度，並在參數遺漏或錯誤時啟動重試機制，以確保最終 Tool Call 的正確性與完整性。

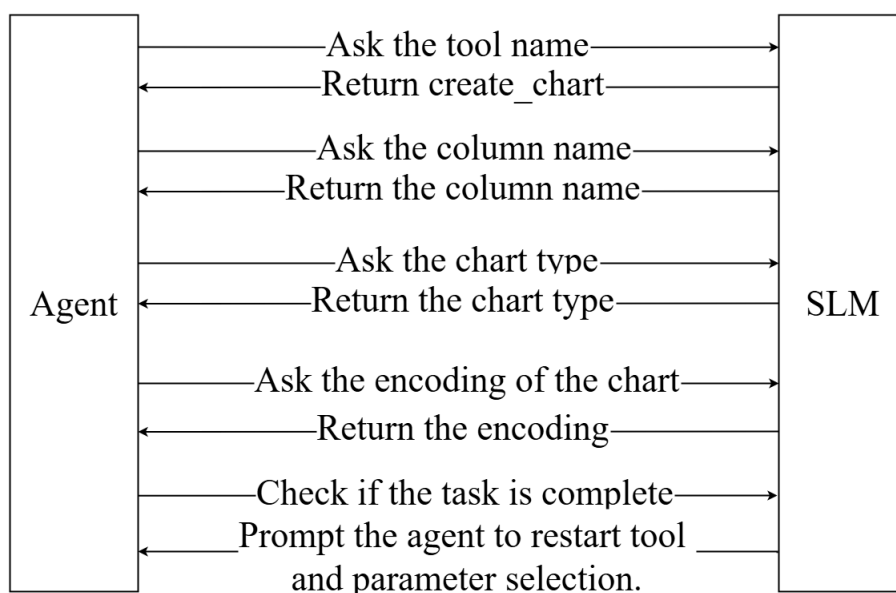


圖 3.5: Tool Call 生成流程圖

3.3.3 指令執行階段

執行階段的整體流程如圖 3.6 所示。當所有工具名稱與參數完成組合後，Agent 會依據產生的 Tool Call 結果，選擇對應工具進行執行。系統會確保透過查詢工具產生合法的資料查詢語句，其中資料來源為 Prometheus，查詢語言則使用 PromQL。當工具產生出對應的 PromQL 查詢語句後，Agent 會將它放進 Grafana 的查詢設定中，作為後續圖表繪製所需的資料來源。之後當使用者開啟圖表或變更查詢條件時，Grafana 就會透過這段語句向 Prometheus 撈取對應資料。接著，圖表設定將轉換為符合 Grafana 規格的格式，並透過上傳工具更新至指定儀表板。最終產出的圖形將即時顯示於前端介面，如圖 3.7 所示。

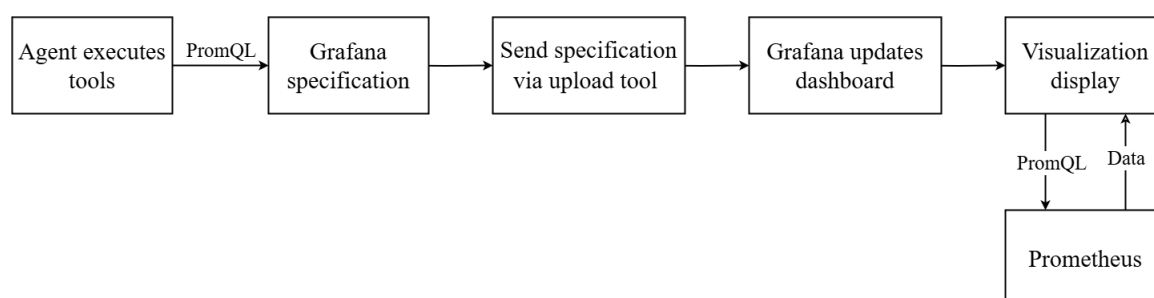


圖 3.6: Agent 執行工具流程圖

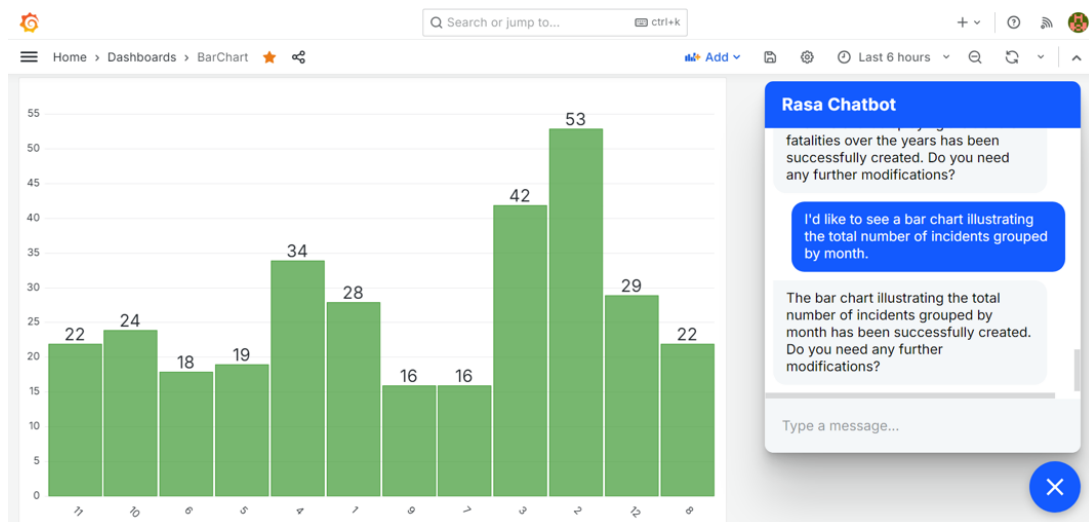


圖 3.7: Grafana 互動介面展示

四、實驗

為驗證本系統在 Tool Call 參數生成的準確性，以及面對使用者輸入不完整時的應對能力，即系統的 Robustness，本研究共設計兩組實驗。第一組實驗採用前述方法章節中介紹的 ChartGPT 公開資料集，評估微調後的 SLM 是否能正確產出各項視覺化參數，並與其他模型進行比較，包含使用通用型資料集訓練的 xLAM [15] 模型以及 ChartGPT [3] 模型。第二組實驗則以本研究於 6G 專案中所使用的私有資料集，模擬使用者的圖表需求，並根據輸入資訊缺漏的程度劃分不同場景，觀察系統是否能如期補足必要資訊並成功完成圖表生成。

4.1 資料集

在實驗一所使用的資料集是 ChartGPT 提供的測試集，包含的資料筆數大約有 2200 筆，詳細的介紹如 3.1.1。在實驗二所使用的是 6G 專案中所使用的私有資料集，來自 WiNES Lab 所釋出的 colosseum-oran-commag 資料集 [23]，我們擷取下行傳輸速率 (tx_brate_downlink) 資料，並聚焦於其中 tr12/exp1 資料夾下的 rome_static_close 場景。該場景中，使用者設備 (User Equipment, UE) 靜止分布於四個基地台 (Base Station, BS) 周圍，並根據所屬的網路切片類型獲得相對應的服務資源配置，以模擬真實中的差異化網路需求。

網路切片 (Network Slicing) 為 5G 及未來 6G 系統中的重要機制，允許營運商依照應用需求切分虛擬網路資源。在此資料集中，網路共分為三種類型：

Slice 0 (eMBB)：強調高頻寬傳輸，適用於影音串流等應用；

Slice 1 (MTC)：適用於物聯網等大規模設備連線任務；

Slice 2 (URLLC)：強調極低延遲，適用於即時控制系統。

在資料結構上，整個場景的資料依據基地台編號分為四個資料夾，命名為 bs1 至 bs4，每個資料夾中皆包含其所服務的 UE 的個別資料檔案，命名格式如 1010123456002_metrics.csv。每筆資料皆記錄某一 UE 在單一時刻的觀測情況與其所屬

切片，如表 4.1 所示。

表 4.1: 資料集所使用到的單一 CSV 檔範例內容

Timestamp	slice_id	tx_brate_downlink
0	0	0.028896
0.25	0	0.017344
0.5	0	0.020608

為了將資料導入視覺化平台，我們於專案中建置了 Prometheus 資料庫，如圖 4.1，並將時間序列資料轉換為符合其接收格式的結構，逐步上傳各時點對應的 tx_brate_downlink 數值。每筆資料皆加註基地台編號、使用者設備編號與網路切片類型等標籤資訊，以利後續查詢與篩選。上述處理後的資料構成 6G 專案中的觀測基礎，亦為本研究實驗二的主要資料來源，用以評估系統在資料不完整情況下的穩定性與圖表生成能力。

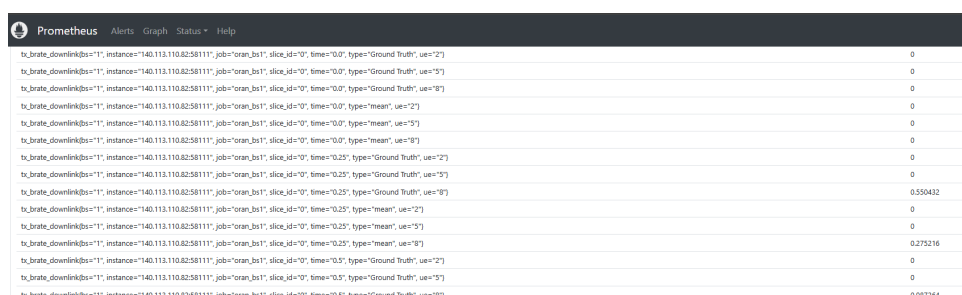


圖 4.1: prometheus 資料庫預覽

4.2 評估指標

為了量化比較各模型在視覺化參數生成任務中的表現，本研究採用兩項常見於機器翻譯領域的評估指標：ROUGE-L 與 BLEU。儘管這兩項指標原本設計用於衡量翻譯的生成品質，但在本研究中，每組視覺化參數皆可組合為具有固定格式的序列，因此可透過序列相似度的比較，作為模型輸出與標準答案之間準確性的評估依據。

4.2.1 ROUGE-L

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation) 是一種以最長公共子序列 (Longest Common Subsequence, LCS) 為基礎的相似度指標，用於衡量預測文字序列與參考文字序列在順序與內容上的重合程度。其 F1-score 計算公式如下：

$$\text{ROUGE-L} = \frac{(1 + \beta^2) \cdot R_{\text{lcs}} \cdot P_{\text{lcs}}}{R_{\text{lcs}} + \beta^2 \cdot P_{\text{lcs}}} \quad (4.1)$$

其中，

$$R_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{|Y|}, \quad P_{\text{lcs}} = \frac{\text{LCS}(X, Y)}{|X|} \quad (4.2)$$

X 為模型生成的文字序列， Y 為參考答案， $\text{LCS}(X, Y)$ 表示 X 與 Y 的最長公共子序列長度， β 為調整 precision 與 recall 權重的參數。

ROUGE-L 的特點在於可在不要求完全連續匹配的前提下，同時衡量模型輸出與參考答案在內容上的重合程度與順序的一致性，因為其計算基礎為 LCS，即在保留文字順序的前提下，兩個序列中所能對齊的最長相同子序列長度。在視覺化任務中，常存在一些被廣泛接受的默認習慣。例如， $x=\text{Time}, y=\text{Value}$ 與 $x=\text{Value}, y=\text{Time}$ 雖然在參數上僅為順序對調，但在視覺化呈現上的意義卻有所不同。人們在閱讀圖表時通常預期時間軸位於 X 軸，若順序錯置，可能導致使用者難以理解圖表內容。這類順序錯誤將使最長公共子序列長度縮短，進而降低 ROUGE-L 的計分結果。因此，ROUGE-L 不僅能評估參數內容的正確性，也能有效反映視覺化邏輯與使用者認知的一致性。

4.2.2 BLEU

BLEU (Bilingual Evaluation Understudy) 同樣是一種常見的機器翻譯評估指標。其基本原理是將生成句子拆解為一系列連續的文字片段，稱為 n-gram，並與參考句子中對應的 n-gram 進行比對，以評估兩者之間的重疊程度。其中， n 表示片段中所包含的字數，例如 1-gram 表示單一詞，2-gram 則為由相鄰兩個詞所組成的片段。n-gram BLEU 的計算方式如下：

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \cdot \log p_n \right) \quad (4.3)$$

其中， p_n 為 n-gram 的精確度， w_n 為加權值，BP 為長度懲罰因子 (brevity penalty, BP)，用以避免模型過度產生簡短輸出。

n-gram 精確度 p_n 定義如下：

$$p_n = \frac{\text{Count}_{\text{clip}}(n\text{-gram})}{\text{Count}(n\text{-gram})} \quad (4.4)$$

其中， $\text{Count}(n\text{-gram})$ 表示模型產生的 n-gram 有多少出現在參考答案中；而 $\text{Count}_{\text{clip}}(n\text{-gram})$ 則限制每個 n-gram 的計數不得超過其在參考答案中出現的次數，

以避免模型透過重複詞語來獲得不實際的高分。例如，若模型輸出中某個詞出現三次，而參考答案中僅出現兩次，則其 $\text{Count}_{\text{clip}}$ 為二，以避免過度重複詞語帶來的不實際加分。

為進一步避免模型僅透過產生簡短片段獲得高分，BLEU 加入了長度懲罰因子，其定義如下：

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases} \quad (4.5)$$

其中， c 為模型輸出的句子長度， r 為參考答案的長度。當模型輸出長度大於參考答案時，BP 為 1，不給予懲罰；反之若模型輸出較短，則會乘上懲罰因子，降低最終得分。

由於 ROUGE-L 強調生成內容與參考答案在順序上的一致性，但在視覺化任務中，部分參數的順序並非絕對。例如前面提到的例子， $x=\text{Time}, y=\text{Value}$ 與 $x=\text{Value}, y=\text{Time}$ 雖與閱讀習慣有差異，卻不一定構成錯誤，類似情況在實際應用中相當常見。為補償 ROUGE-L 對順序過於敏感的限制，本研究引入 1-gram BLEU 作為輔助指標，忽略順序因素，聚焦於參數本身是否正確地出現在模型輸出中。

4.3 實驗一

4.3.1 實驗流程

本研究將六個視覺化參數依照固定順序串接為一段文字序列，並以空格分隔各項參數。舉例而言，以下為某筆資料中六個視覺化參數的生成結果：

- 欄位選擇 (Select columns) : Rank
- 篩選條件 (Add filter) : none
- 聚合函數 (Add aggregations) : count
- 圖表類型 (Select chart type) : arc
- 視覺編碼 (Choose encoding) : $x=\text{Rank}, y=\text{count_Rank}, \text{color}=\text{none}$
- 排序設定 (Add sort) : none

則其對應的序列為：

Rank none count arc x=Rank,y=count_Rank,color=none none

而在本研究所提出的 Tool Call 方法中，六個視覺化參數被分配至五個工具中。為進行評估，我們會從模型生成的 Tool Call 中剷除工具名稱，僅保留實際使用到的參數，並依照前述順序（欄位選擇、篩選條件、聚合函數、圖表類型、視覺編碼與排序）組合為一段固定長度的文字序列。若某項參數在生成過程中未被指定，則會以 none 進行補足，以確保各筆資料皆具六個欄位。

同樣地，我們亦將此工具清單提供給 xLAM [15] 模型，並以相同方式處理其輸出結果，統一格式後計算 ROUGE-L 與 BLEU 分數。藉此確保各模型皆在一致條件下進行比較，準確反映其在參數生成任務上的表現差異。

4.3.2 結果分析

表 4.2 彙整了不同模型在 ChartGPT 測試資料集上的參數生成表現。我們以 ROUGE-L 與 BLEU 作為主要評估指標，分別對應參數生成的順序性與內容覆蓋率。由於原始的 Phi3.5 並不具備 Tool Call 能力，無法穩定生成結構化的參數輸出，因此本研究選擇其他具備 Tool Call 輸出能力的通用語言模型作為基準模型（baseline），包含 xLAM、LLaMA3.2 與 Qwen2.5。這些模型雖未經過視覺化任務微調，但皆可支援工具選擇與參數格式的生成，故被納入比較。

相比之下，經過視覺化任務微調的模型，包括 ChartGPT 與本研究所使用之 Phi3.5，在兩項指標上皆顯著優於基準模型。本研究模型於 ROUGE-L 指標上達 0.88，略高於 ChartGPT 的 0.87，顯示其在生成順序正確性的層面上更具穩定性；BLEU 得分則與 ChartGPT 持平，皆為 0.91，反映兩者在參數涵蓋率上的表現相當。

雖然本系統所使用模型參數量略高於 ChartGPT，但本研究特別設計逐步參數生成流程，並導入 Tool Call 機制，實現更高的系統彈性。此設計將在下一節實驗二中進一步驗證其於使用者輸入不完整時的 Robustness。

表 4.2: 不同模型於 ChartGPT 測試集上之參數生成表現

Method	Model	Model Size	ROUGE-L	BLEU
Baseline	xLAM	7B	0.41	0.54
	LLaMA3.2	3B	0.50	0.63
	Qwen3	8B	0.48	0.51
	Phi4-mini	3.8B	0.39	0.42
	Gemma3	4B	0.44	0.50
Fine-tuned	ChartGPT	2.85B	0.87	0.91
	Ours	3.8B	0.88	0.91

4.4 實驗二

4.4.1 使用場景介紹

本實驗設計四個層級的使用場景，模擬使用者在查詢圖表時提供資訊的完整程度，分別針對查詢語句 (Q_i) 與結構化資料內容 (C_i) 進行劃分。 Q_i 表示使用者輸入中是否明確包含圖表類型， C_i 則表示結構化資料是否提供足夠資訊以供圖表生成。

其中 C_i 可細分為：

- C_1 ：欄位名稱集合
- C_2 ：欄位類型集合
- C_3 ：資料表名稱
- C_4 ：資料範例集合

本研究依據上述資訊是否提供，定義四個輸入場景層級，如表 4.3 所示。由 Level 1 到 Level 4，使用者所提供的訊息逐漸減少，Level 4 為完全未提供圖表類型與結構資訊的最困難場景。

表 4.3: Robustness 測試中四種輸入資訊完整度場景

Level	Q_i (Contains Chart Type)	C_i (Structured Data)
Level 1	✓	C_{1-4}
Level 2	✗	C_{1-4}
Level 3	✗	C_1
Level 4	✗	✗

4.4.2 實驗流程

由於 ChartGPT 並非以 Tool Call 為目標進行訓練，其輸出結果為一組六項視覺化參數（欄位選擇、篩選條件、聚合函數、圖表類型、視覺編碼、排序設定）。為使其結果可與其他 Tool Call 模型進行比較，本研究將 ChartGPT 輸出內容轉換為 Vega-Lite 格式，並使用 Python 中的 matplotlib 套件繪製對應圖表。

ChartGPT 的視覺化轉換流程如下：

1. 將使用者語句與結構化資料格式化為 ChartGPT 可理解的 prompt。
2. 按照六個視覺化任務步驟逐一詢問模型，並記錄每次輸出的參數。
3. 將生成的六個參數整理為字典格式，並轉換為 Vega-Lite 規格。
4. 利用 Python 程式將 Vega-Lite 規格繪製成對應圖表。

與此同時，本研究方法亦使用相同的輸入語句與結構化資料作為起點，經由 SLM 與 Agent 組合生成完整的 Tool Call，並透過 Grafana 視覺化平台實際繪製圖表。最終將 ChartGPT 所繪製之圖形與 Grafana 生成結果進行並列比對，以評估兩個系統在不同資訊完整度下的強健性。

4.4.3 實驗結果比較

4.4.3.1 Level 1：查詢語句與資料完整輸入

使用者語句 (Q_i)：

Create a **line chart** for tx_brate_downlink over all timestamps.

結構化資料 (D_i)：

```
5g_slice_performance
<head> timestamp, slice_id, tx_brate_downlink
<type> quantitative, nominal, quantitative
<data> 0, 1, 0.0288 <line> 0.25, 1, 0.0173 <line>
```

圖 4.2 比較了現有系統與本研究所產出圖表在視覺呈現上的差異。右圖為本研究生成之圖表，除了明確標示圖例與資料欄位名稱 tx_brate_downlink 外，在 x 軸刻度的設計上也更為細緻，使得數值變化與峰值分佈更容易被肉眼辨識。

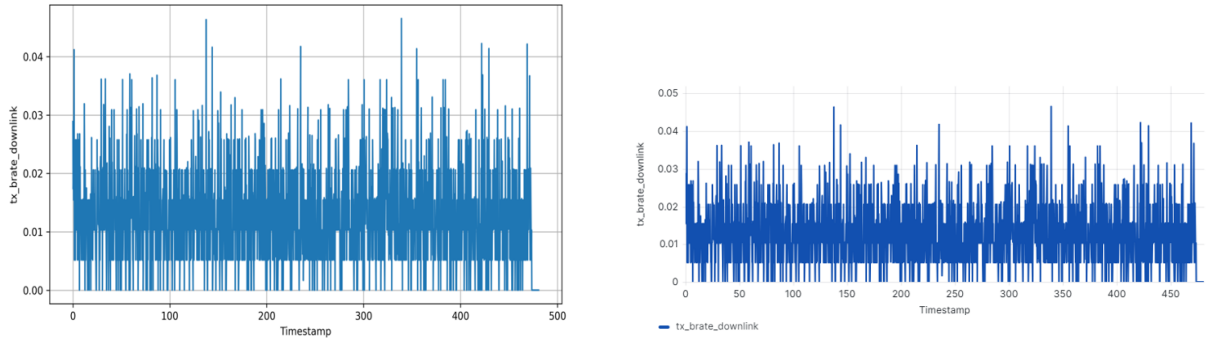


圖 4.2: Level 1 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法

4.4.3.2 Level 2：查詢語句缺少圖表類型

使用者語句 (Q_i):

Show tx_brat_downlink for all timestamps.

結構化資料 (D_i):

5g_slice_performance

<head> timestamp, slice_id, tx_brat_downlink

<type> quantitative, nominal, quantitative

<data> 0, 1, 0.0288 <line> 0.25, 1, 0.0173 <line>

在 Level 2 場景中，使用者輸入中未明確指定圖表類型，考驗系統對語意的理解與預設圖表選擇策略。結果顯示，無論是 ChartGPT 或本研究方法，皆自動判定以折線圖呈現 tx_brat_downlink 指標，顯示兩者在此場景下的圖表判斷行為一致。此外，從圖 4.3 可見，兩系統所產出的圖表在座標軸設定、圖例標示與整體視覺結構上亦相近，差異不大。

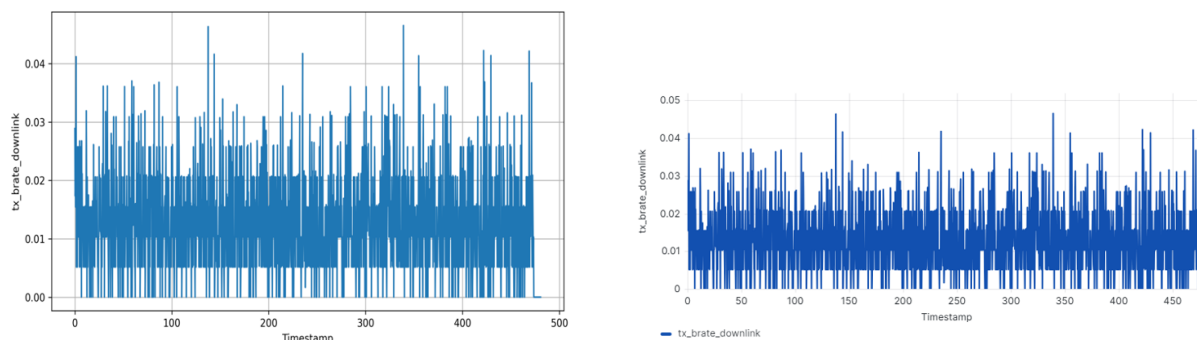


圖 4.3: Level 2 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法

4.4.3.3 Level 3：資料僅提供欄位集合

使用者語句 (Q_i):

Show tx_brat_downlink for all timestamps.

結構化資料 (D_i):

<head> timestamp, slice_id, tx_brat_downlink

在圖 4.4 的 Level 3 的場景中，使用者僅提供最基本的資料欄位集合作為輸入，缺乏明確的圖表需求。儘管 ChartGPT 的訓練資料中未涵蓋此類輸入場景，其系統仍能根據可用欄位推論並產生對應圖表。同樣地，本研究所提出的方法亦能在此條件下成功生成圖表，並透過 Agent 機制進一步確認資料來源與欄位的正確性。儘管整體設定大致相同，但由於測試資料涵蓋的時間軸範圍較廣，本研究所生成的圖表在 x 軸上呈現出較為密集的刻度標示，進而有助於使用者更精確地觀察並辨識資料中的峰值位置。

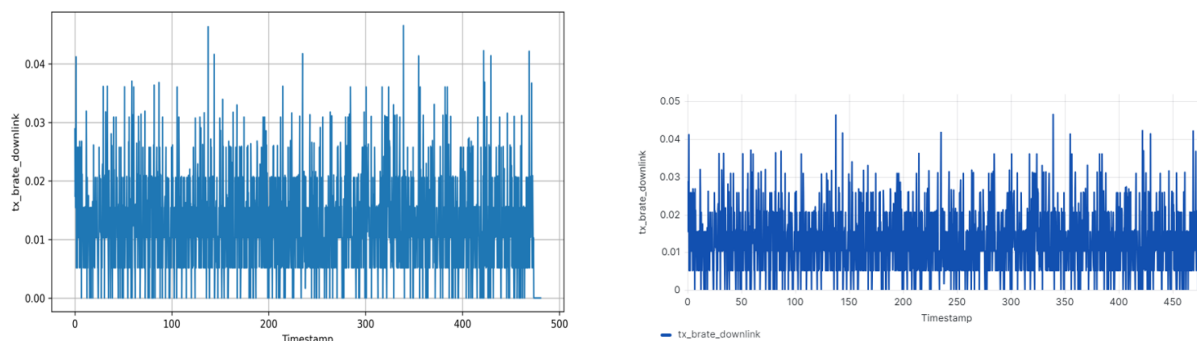


圖 4.4: Level 3 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法

4.4.3.4 Level 4：資料並未標註欄位資訊

在此場景中，使用者提供的資料缺乏欄位名稱、欄位類型等結構化資訊，無法讓 SLM 明確判斷可用的欄位集合。然而，自然語言輸入中仍提及欲視覺化的指標名稱，使模型需僅憑語句推理其對應欄位，完成視覺化任務。

使用者語句 (Q_i)：

Show tx_brat_e_downlink for all timestamps.

結構化資料 (D_i)： X

在圖 4.5 的 Level 4 的場景中，由於資料中未明確列出所有欄位資訊，關鍵欄位僅出現在使用者語句中，ChartGPT 雖成功推論 tx_brat_e_downlink 作為 Y 軸欄位，卻未能正確辨識 timestamp 作為時間軸，導致無法依據欄位名稱正確擷取時間序列資料，進而無法生成正確的圖表。相較之下，本研究所提出的方法透過 Agent 的輸入驗證機制進行修正，即使在欄位標註不完整的情況下，仍能正確解析資料並生成合理的圖表。

```
{
  "mark": "line",
  "encoding": {
    "x": {
      "field": "timestamp",
      "type": "quantitative"
    },
    "y": {
      "field": "tx_brat_e_downlink",
      "type": "quantitative"
    }
  }
}
```

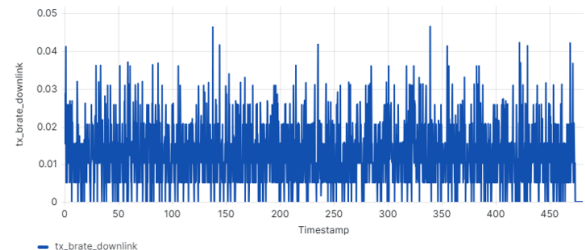


圖 4.5: Level 4 場景下圖表生成結果之比較，左圖為 ChartGPT，右圖為我們的方法

4.4.4 實驗二討論

實驗結果顯示，ChartGPT 在資訊較完整的 Level 1 至 Level 3 場景下皆能生成與本系統相似的圖表，顯示其即使未針對缺失場景進行微調，模型本身仍具備一定的適應能力 (adaptability)。然而在 Level 4 當中，由於完全缺乏結構化資料支援，ChartGPT 無法正確生成圖表，其對應的 Vega-Lite JSON 結構亦無法完成語意合理的欄位對應。

為確認圖表生成結果的合理性，本研究採取人工方式進行檢查，主要聚焦於資料值的對應關係與圖形趨勢的視覺一致性。透過觀察幾個具代表性的時間點，我們可判斷圖形是否與原始資料一致。例如，在 timestamp 約為 150 與 350 的區段通常會出現明顯的峰值變化，而在時間軸尾端，tx_brake_downlink 常會下降至 0。這些特徵成為我們在判讀圖表正確性時的重要參考依據。

透過上述方法可確認在 Level 1 至 Level 3 中，ChartGPT 與我們系統所生成圖形皆具備相似趨勢與正確性，然而 Level 4 中則出現明顯錯誤，無法正確繪製圖表。

五、 總結

5.1 結論

本研究提出一套由 SLM 驅動的 Agent 系統來生成與執行 Tool call，並以 Grafana 為例進行視覺化生成。透過引入 Agent 機制，系統能夠將 SLM 分別產生的工具名稱與參數組合成完整的 Tool Call，進而執行前端圖表生成、後端資料查詢與資訊補足等任務，提升視覺化流程的彈性與穩定性。

實驗一採用 ChartGPT 公開資料集驗證本系統於參數生成任務的效能。結果顯示，我們微調後的 Phi-3.5 模型在 ROUGE-L 指標上達到 0.88，優於同為 SLM 系統的 ChartGPT (0.87)，在 BLEU 指標上則與其持平 (0.91)，證明本方法在視覺化參數的順序性與涵蓋性上具備穩定且精確的表現。

進一步地，實驗一還比較了本模型與三個未經視覺化任務微調的一般化語言模型 (xLAM、LLaMA3.2 與 Qwen2.5)，結果顯示本模型在 ROUGE-L 指標上平均提升約 78%，在 BLEU 指標上亦有約 48% 的提升，突顯微調對於視覺化任務的重要性。

實驗二進一步探討系統於資訊不完整情境下的強健性 (Robustness)，針對不同層級的輸入資訊缺漏情境進行測試。結果顯示，ChartGPT 在前三種情境中多能與本研究方法產出相似的圖表結果，顯示其具備一定程度的適應性。然而在最困難的 Level 4 場景中，ChartGPT 無法成功推理出視覺化目標，導致圖表繪製失敗；反觀本研究方法仍能完成合理圖表生成，驗證我們的 Tool Call 流程設計能有效提升系統於實務應用中的容錯性與穩健性。

5.2 未來展望

本研究目前採用逐步生成的方式引導 SLM 產出 Tool Call 所需的內容，並透過 Agent 協助組裝工具名稱與參數。未來可進一步朝向使模型能夠一次性輸出結構完整、語法正確的 Tool Call，進一步簡化 Agent 流程，提升系統整體回應速度。

此外，本研究所設計的 Tool Call 機制具備高度擴展性，未來不僅可應用於資料視覺化領域，也能結合其他任務導向的應用場景。近期興起的模型上下文協定（Model Context Protocol, MCP [24]）提供了一種統一存取外部 API 的管道：MCP 伺服器維護一組帶有結構化註解的 API，AI Agent 可根據任務需求與 API 的功能說明，自主選擇適當的工具進行調用。透過此架構，AI Agent 不僅能執行單一平台（如 Grafana）的操作，亦能拓展至多種外部系統之整合，實現跨平台與多工具整合的能力。結合 Tool Call 機制與 MCP 架構的未來系統，有望進一步提升整個系統的自主決策與工具調用能力，朝向更加通用且可擴展的 AI 自動化平台邁進。

六、 參考文獻

- [1] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, *et al.*, “A survey of large language models,” *arXiv preprint arXiv:2303.18223*, vol. 1, no. 2, 2023.
- [2] L. Shen, E. Shen, Y. Luo, X. Yang, X. Hu, X. Zhang, Z. Tai, and J. Wang, “Towards natural language interfaces for data visualization: A survey,” *IEEE transactions on visualization and computer graphics*, vol. 29, no. 6, pp. 3121–3144, 2022.
- [3] Y. Tian, W. Cui, D. Deng, X. Yi, Y. Yang, H. Zhang, and Y. Wu, “Chartgpt: Leveraging llms to generate charts from abstract natural language,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 3, pp. 1731–1745, 2025.
- [4] T. Masterman, S. Besen, M. Sawtell, and A. Chao, “The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey,” *arXiv preprint*, 2024. arXiv:2404.11584.
- [5] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, *et al.*, “A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness,” *arXiv preprint arXiv:2411.03350*, 2024.
- [6] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, J. Tan, W. Yao, Z. Liu, Y. Feng, R. RN, *et al.*, “Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 54463–54482, 2024.
- [7] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024.
- [8] A. Van Dam, “Post-wimp user interfaces,” *Communications of the ACM*, vol. 40, no. 2, pp. 63–67, 1997.
- [9] P. Maddigan and T. Susnjak, “Chat2vis: Generating data visualizations via natural language using chatgpt, codex and gpt-3 large language models,” *IEEE Access*, vol. 11, pp. 45181–45193, 2023.
- [10] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, “A prompt pattern catalog to enhance prompt engineering with chatgpt,” *arXiv preprint arXiv:2302.11382*, 2023.
- [11] P. Vaithilingam, E. L. Glassman, J. P. Inala, and C. Wang, “Dynavis: Dynamically synthesized ui widgets for visualization editing,” in *Proceedings of the 2024 CHI Conference on Human Factors*

- in Computing Systems*, CHI '24, (New York, NY, USA), Association for Computing Machinery, 2024.
- [12] Z. Yang, Z. Zhou, S. Wang, X. Cong, X. Han, Y. Yan, Z. Liu, Z. Tan, P. Liu, D. Yu, *et al.*, “Matplotagent: Method and evaluation for llm-based agentic scientific data visualization,” *arXiv preprint arXiv:2402.11453*, 2024.
 - [13] G. Ouyang, J. Chen, Z. Nie, Y. Gui, Y. Wan, H. Zhang, and D. Chen, “nvagent: Automated data visualization from natural language via collaborative agent workflow,” *arXiv preprint arXiv:2502.05036*, 2025.
 - [14] L. Podo, M. Angelini, and P. Velardi, “V-recs, a low-cost llm4vis recommender with explanations, captioning and suggestions,” *arXiv preprint arXiv:2406.15259*, 2024.
 - [15] J. Zhang, T. Lan, M. Zhu, Z. Liu, T. Hoang, S. Kokane, W. Yao, J. Tan, A. Prabhakar, H. Chen, *et al.*, “xlam: A family of large action models to empower ai agent systems,” *arXiv preprint arXiv:2409.03215*, 2024.
 - [16] G. Dong, H. Yuan, K. Lu, C. Li, M. Xue, D. Liu, W. Wang, Z. Yuan, C. Zhou, and J. Zhou, “How abilities in large language models are affected by supervised fine-tuning data composition,” *arXiv preprint arXiv:2310.05492*, 2023.
 - [17] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 53728–53741, 2023.
 - [18] W. Liu, X. Huang, X. Zeng, X. Hao, S. Yu, D. Li, S. Wang, W. Gan, Z. Liu, Y. Yu, *et al.*, “Toolace: Winning the points of llm function calling,” *arXiv preprint arXiv:2409.00920*, 2024.
 - [19] J. Huang, H. Chen, F. Yu, and W. Lu, “From detection to application: Recent advances in understanding scientific tables and figures,” *ACM Computing Surveys*, vol. 56, no. 10, pp. 1–39, 2024.
 - [20] L. Shen, E. Shen, Y. Luo, X. Yang, X. Hu, X. Zhang, Z. Tai, and J. Wang, “Towards natural language interfaces for data visualization: A survey,” *IEEE transactions on visualization and computer graphics*, vol. 29, no. 6, pp. 3121–3144, 2022.
 - [21] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024.
 - [22] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
 - [23] L. Bonati, S. D’Oro, M. Polese, S. Basagni, and T. Melodia, “Intelligence and learning in o-ran for data-driven nextg cellular networks,” *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021.
 - [24] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” *arXiv preprint arXiv:2503.23278*, 2025.